

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 1-2

Студент группы *ИВБО-20-24 Лияскин.В.А.*

(подпись)

Ассистент *Шерри.Г.Д.*

(подпись)

Отчет представлен «19» сентября 2026 г.

Москва 2026 г.

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 1

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

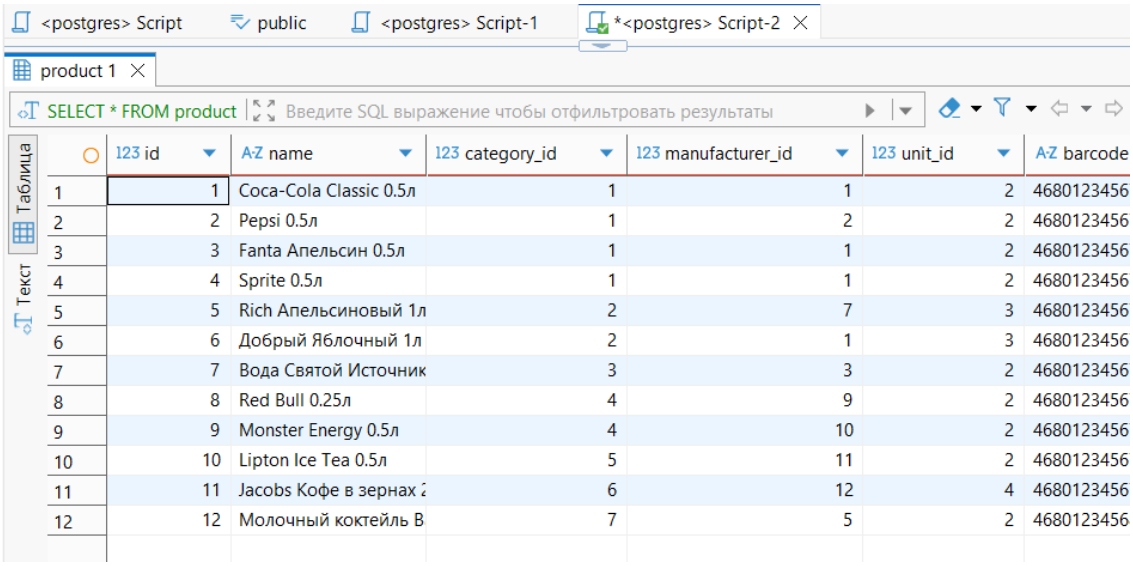
Отчет представлен «12»сентября 2026 г.

Москва 2026 г.

Задание №1

1.1. Выбор всех столбцов (*)

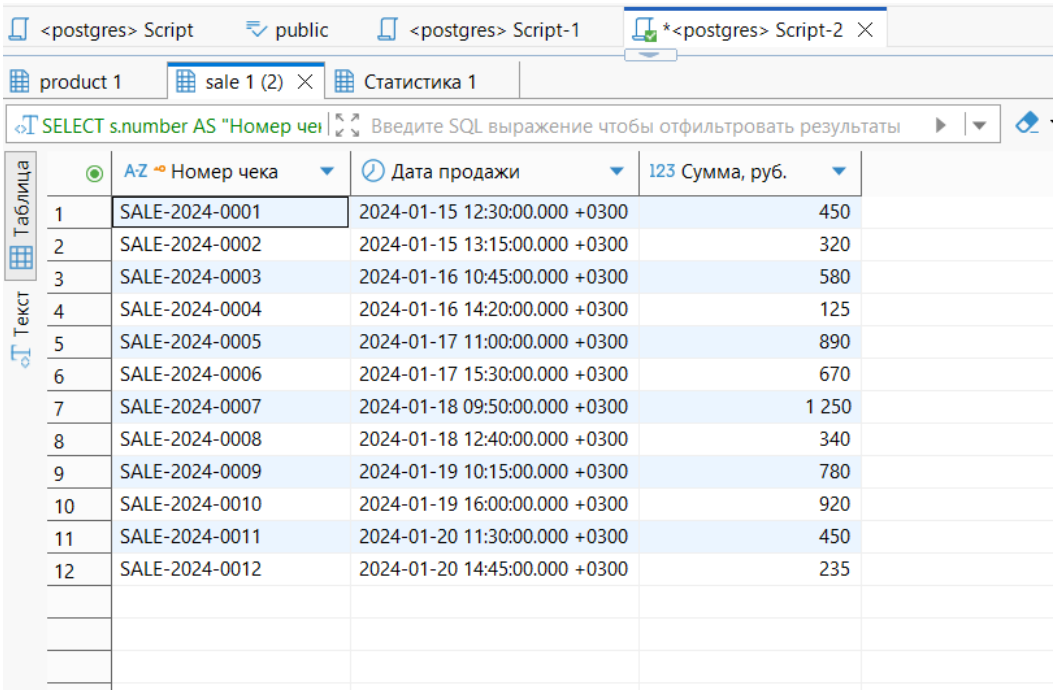
```sql
SELECT * FROM product;
```



	id	name	category_id	manufacturer_id	unit_id	barcode
1	1	Coca-Cola Classic 0.5л	1	1	2	4680123456
2	2	Pepsi 0.5л	1	2	2	4680123456
3	3	Fanta Апельсин 0.5л	1	1	2	4680123456
4	4	Sprite 0.5л	1	1	2	4680123456
5	5	Rich Апельсиновый 1л	2	7	3	4680123456
6	6	Добрый Яблочный 1л	2	1	3	4680123456
7	7	Вода Святой Источник	3	3	2	4680123456
8	8	Red Bull 0.25л	4	9	2	4680123456
9	9	Monster Energy 0.5л	4	10	2	4680123456
10	10	Lipton Ice Tea 0.5л	5	11	2	4680123456
11	11	Jacobs Кофе в зернах	6	12	4	4680123456
12	12	Молочный коктейль В	7	5	2	4680123456

1.2. Выбор конкретных полей с псевдонимом (AS)

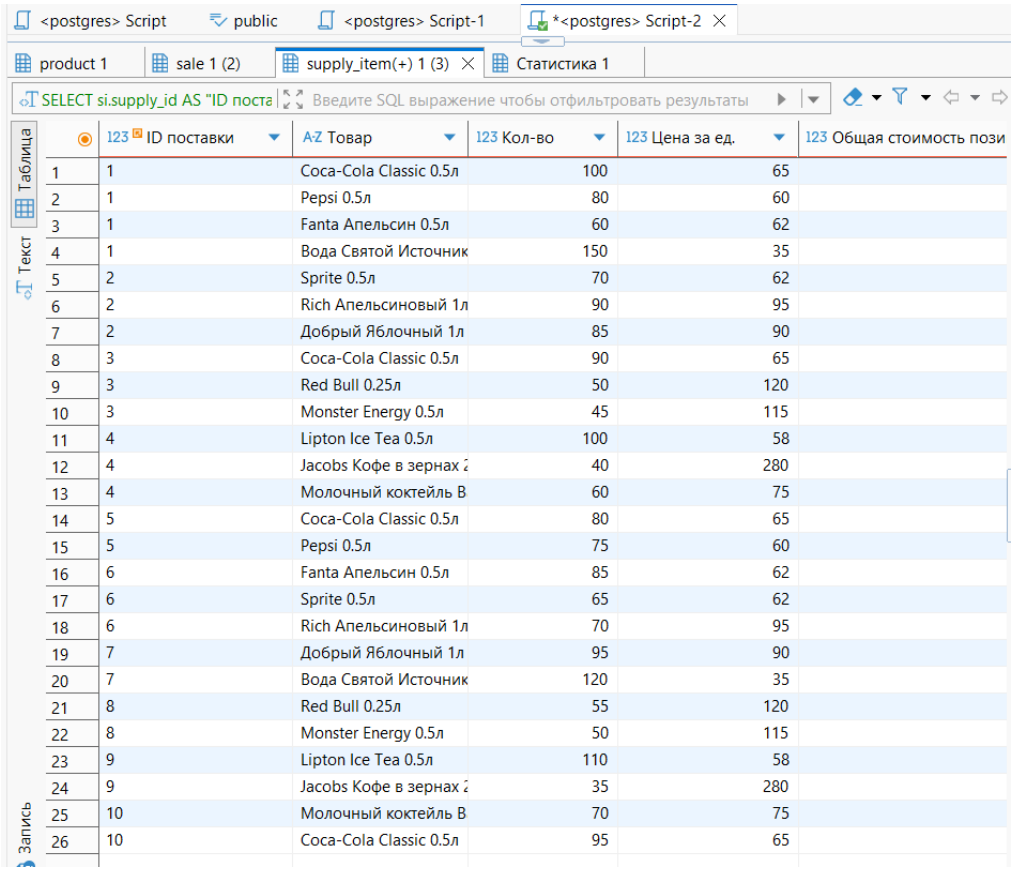
```sql
SELECT
    s.number AS "Номер чека",
    s.date AS "Дата продажи",
    s.total_amount AS "Сумма, руб."
FROM sale s;
```



	Номер чека	Дата продажи	Сумма, руб.
1	SALE-2024-0001	2024-01-15 12:30:00.000 +0300	450
2	SALE-2024-0002	2024-01-15 13:15:00.000 +0300	320
3	SALE-2024-0003	2024-01-16 10:45:00.000 +0300	580
4	SALE-2024-0004	2024-01-16 14:20:00.000 +0300	125
5	SALE-2024-0005	2024-01-17 11:00:00.000 +0300	890
6	SALE-2024-0006	2024-01-17 15:30:00.000 +0300	670
7	SALE-2024-0007	2024-01-18 09:50:00.000 +0300	1 250
8	SALE-2024-0008	2024-01-18 12:40:00.000 +0300	340
9	SALE-2024-0009	2024-01-19 10:15:00.000 +0300	780
10	SALE-2024-0010	2024-01-19 16:00:00.000 +0300	920
11	SALE-2024-0011	2024-01-20 11:30:00.000 +0300	450
12	SALE-2024-0012	2024-01-20 14:45:00.000 +0300	235

1.3. Математическое выражение в списке выборки

```sql
SELECT
    si.supply_id AS "ID поставки",
    p.name AS "Товар",
    si.quantity AS "Кол-во",
    si.price AS "Цена за ед.",
    si.quantity * si.price AS "Общая стоимость позиции"
FROM supply_item si
JOIN product p ON si.product_id = p.id;
```



<postgres> Script | public | <postgres> Script-1 | *<postgres> Script-2

product 1 | sale 1 (2) | supply_item(+) 1 (3) | Статистика 1

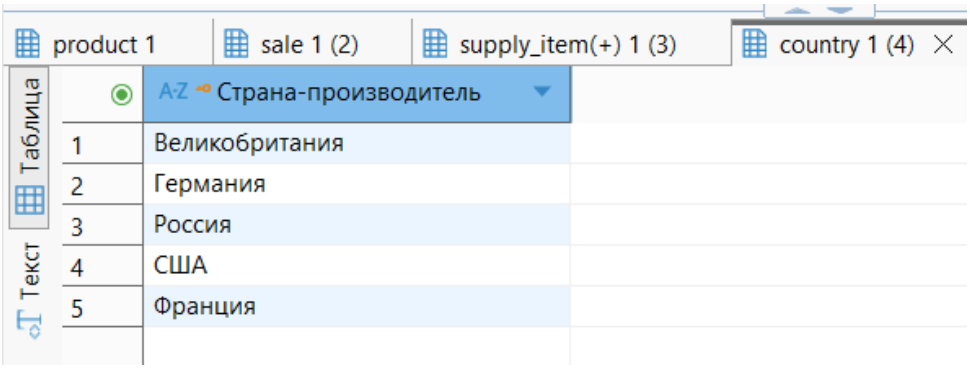
SELECT si.supply_id AS "ID поста | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	123 ID поставки	A-Z Товар	123 Кол-во	123 Цена за ед.	123 Общая стоимость пози
1	1	Coca-Cola Classic 0.5л	100	65	
2	1	Pepsi 0.5л	80	60	
3	1	Fanta Апельсин 0.5л	60	62	
4	1	Вода Святой Источник	150	35	
5	2	Sprite 0.5л	70	62	
6	2	Rich Апельсиновый 1л	90	95	
7	2	Добрый Яблочный 1л	85	90	
8	3	Coca-Cola Classic 0.5л	90	65	
9	3	Red Bull 0.25л	50	120	
10	3	Monster Energy 0.5л	45	115	
11	4	Lipton Ice Tea 0.5л	100	58	
12	4	Jacobs Кофе в зернах 2	40	280	
13	4	Молочный коктейль В	60	75	
14	5	Coca-Cola Classic 0.5л	80	65	
15	5	Pepsi 0.5л	75	60	
16	6	Fanta Апельсин 0.5л	85	62	
17	6	Sprite 0.5л	65	62	
18	6	Rich Апельсиновый 1л	70	95	
19	7	Добрый Яблочный 1л	95	90	
20	7	Вода Святой Источник	120	35	
21	8	Red Bull 0.25л	55	120	
22	8	Monster Energy 0.5л	50	115	
23	9	Lipton Ice Tea 0.5л	110	58	
24	9	Jacobs Кофе в зернах 2	35	280	
25	10	Молочный коктейль В	70	75	
26	10	Coca-Cola Classic 0.5л	95	65	

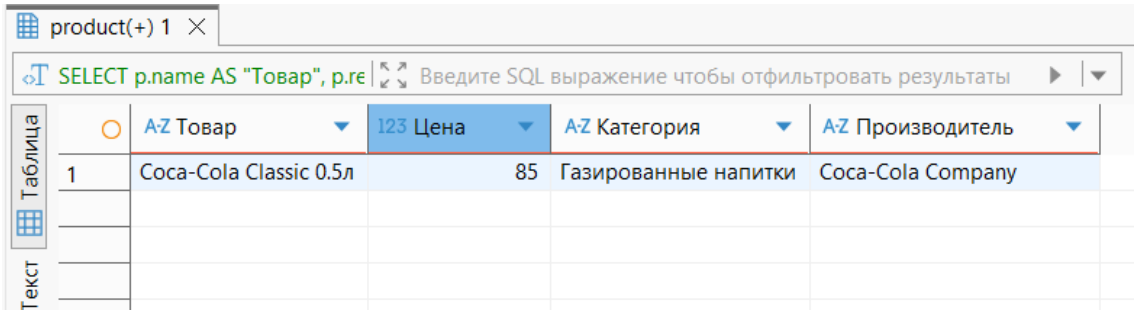
1.4. Удаление дубликатов (DISTINCT)

```sql
SELECT DISTINCT c.name AS "Страна-производитель"
FROM manufacturer m
JOIN country c ON m.country_id = c.id
ORDER BY c.name;
```



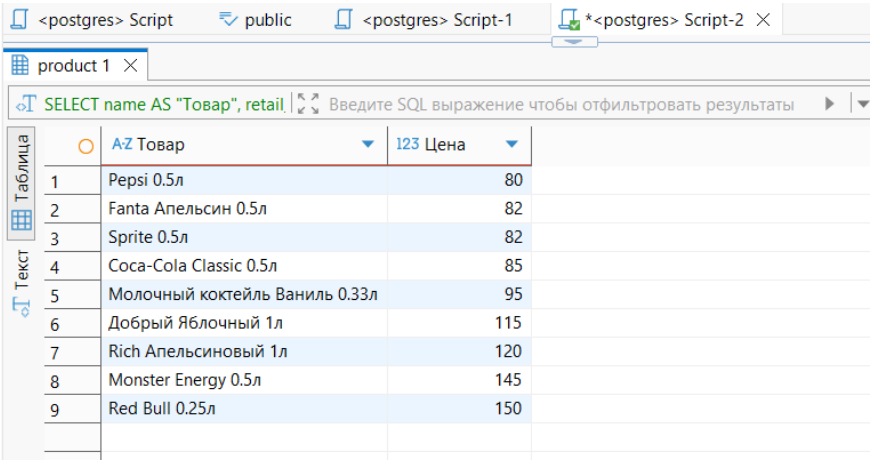
1.5. Логические связки AND и OR

```sql
SELECT
    p.name AS "Товар",
    p.retail_price AS "Цена",
    c.name AS "Категория",
    m.name AS "Производитель"
FROM product p
JOIN category c ON p.category_id = c.id
JOIN manufacturer m ON p.manufacturer_id = m.id
WHERE (p.name LIKE '%Coca-Cola%' OR p.name LIKE '%Pepsi%')
  AND p.retail_price > 80;
```



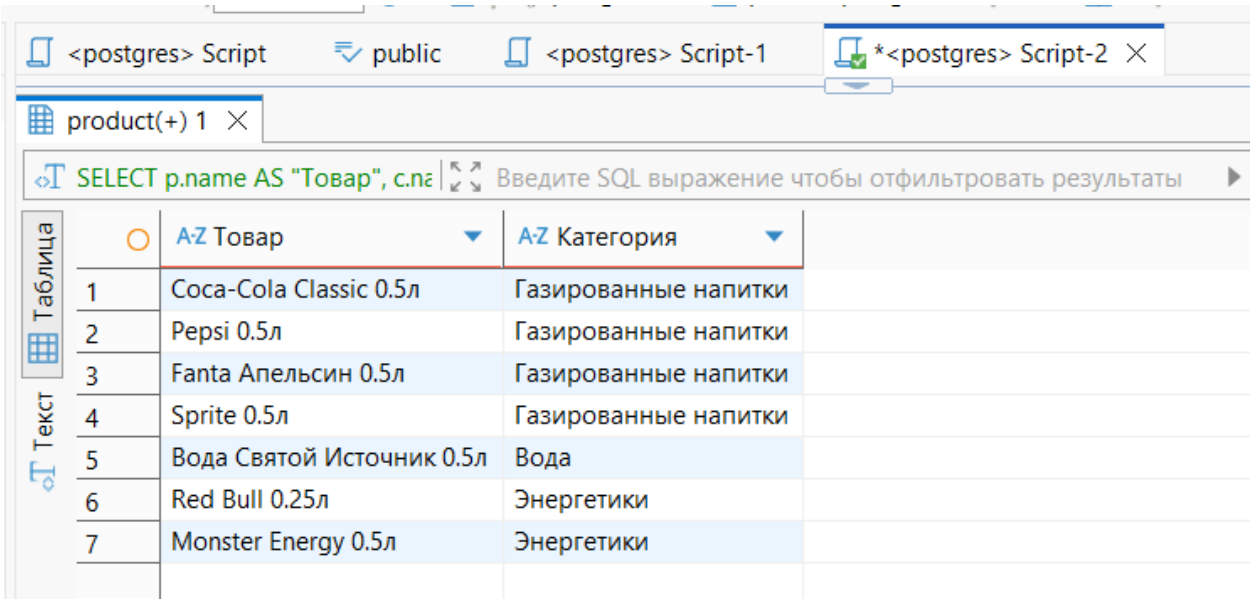
1.6. Проверка принадлежности диапазону (BETWEEN)

```sql
SELECT
    name AS "Товар",
    retail_price AS "Цена"
FROM product
WHERE retail_price BETWEEN 80 AND 150
ORDER BY retail_price;
```



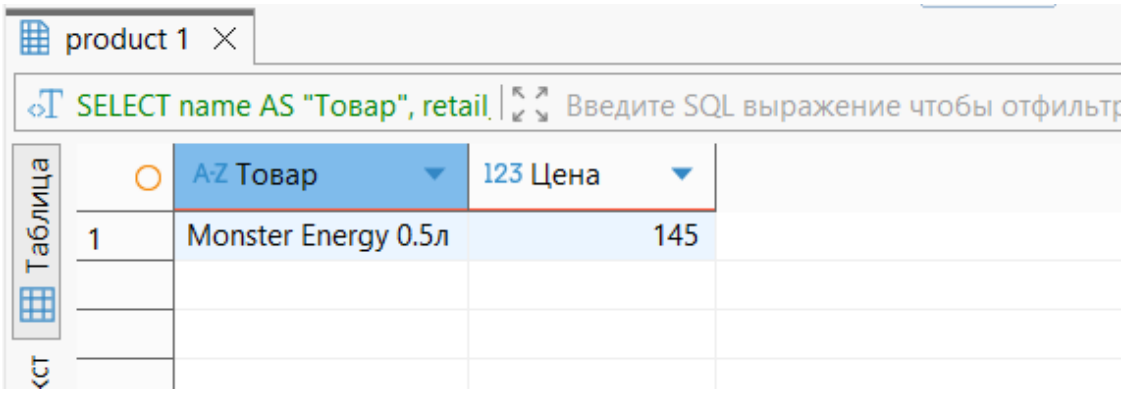
1.7. Проверка вхождения в множество (IN)

```sql
SELECT
    p.name AS "Товар",
    c.name AS "Категория"
FROM product p
JOIN category c ON p.category_id = c.id
WHERE c.name IN ('Газированные напитки', 'Энергетики', 'Вода');
```



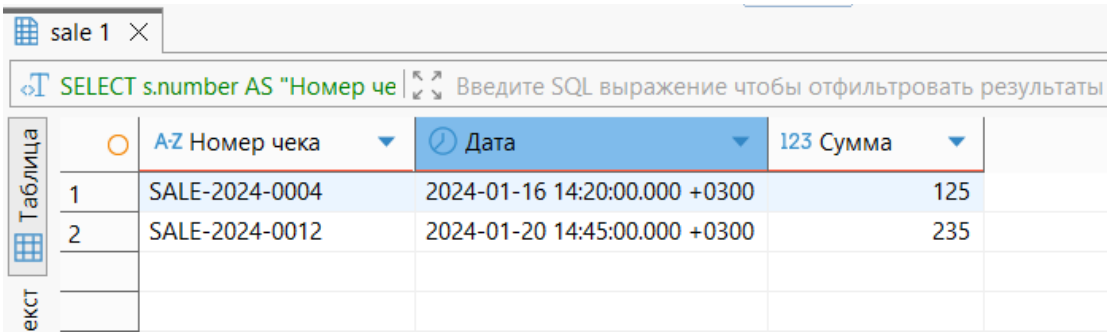
1.8. Сравнение с шаблоном (LIKE)

```sql
SELECT
    name AS "Товар",
    retail_price AS "Цена"
FROM product
WHERE name LIKE '%Energy%';
```



1.9. Проверка на NULL (IS NULL / IS NOT NULL)

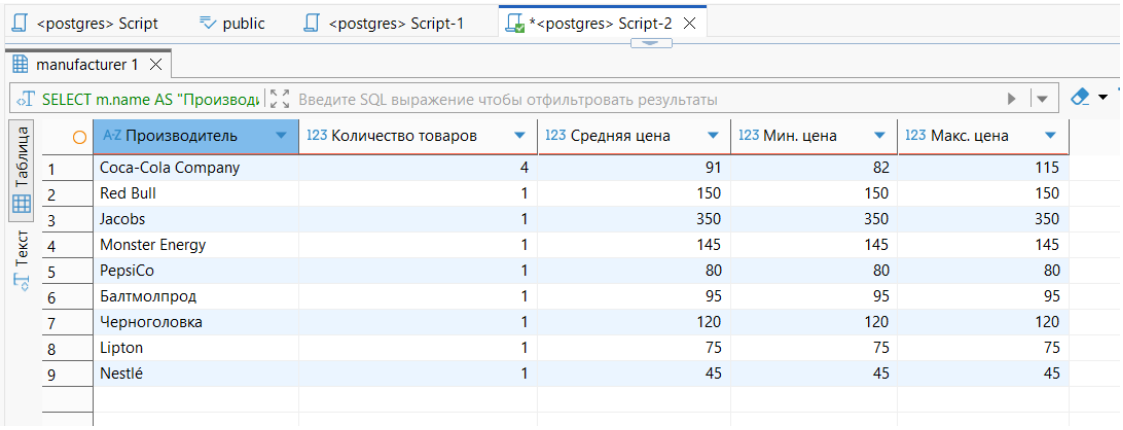
```sql
SELECT
    s.number AS "Номер чека",
    s.date AS "Дата",
    s.total_amount AS "Сумма"
FROM sale s
WHERE s.client_id IS NULL;
```



Задание №2

2.1. Группировка + агрегатные функции

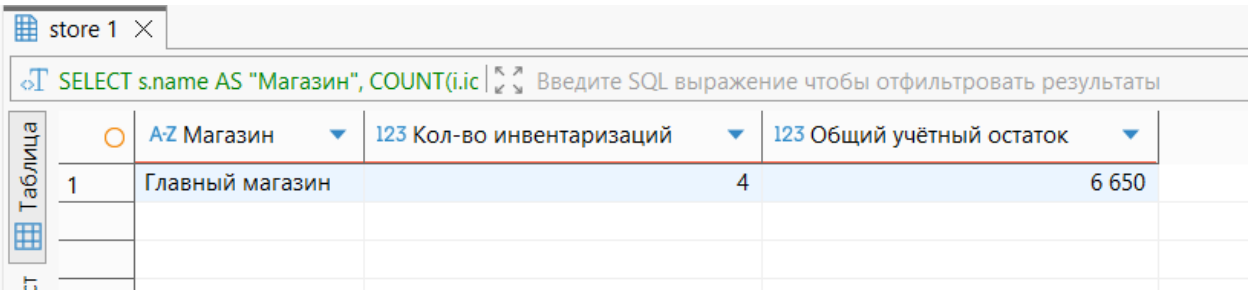
```sql
SELECT
    m.name AS "Производитель",
    COUNT(*) AS "Количество товаров",
    ROUND(AVG(p.retail_price), 2) AS "Средняя цена",
    MIN(p.retail_price) AS "Мин. цена",
    MAX(p.retail_price) AS "Макс. цена"
FROM product p
JOIN manufacturer m ON p.manufacturer_id = m.id
GROUP BY m.name
ORDER BY COUNT(*) DESC;
```



	Производитель	Количество товаров	Средняя цена	Мин. цена	Макс. цена
1	Coca-Cola Company	4	91	82	115
2	Red Bull	1	150	150	150
3	Jacobs	1	350	350	350
4	Monster Energy	1	145	145	145
5	PepsiCo	1	80	80	80
6	Балтмолпрод	1	95	95	95
7	Черноголовка	1	120	120	120
8	Lipton	1	75	75	75
9	Nestlé	1	45	45	45

2.2. HAVING + ORDER BY

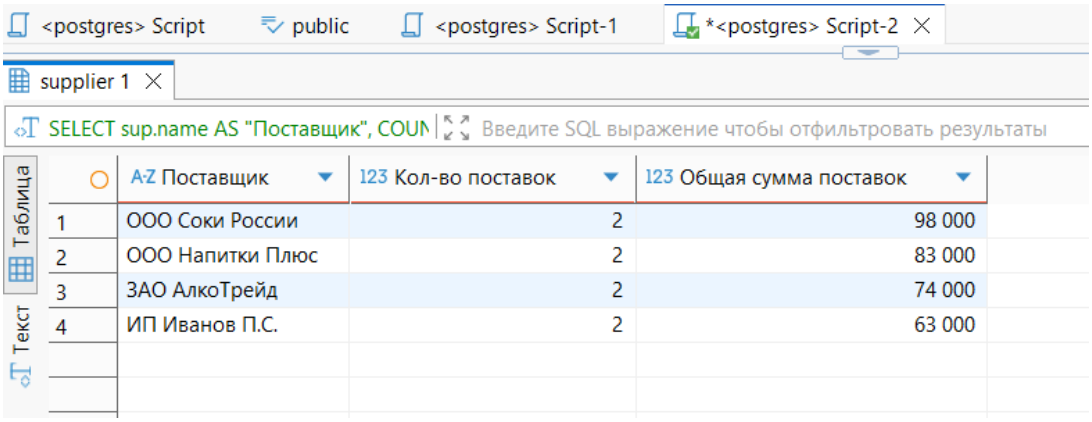
```sql
SELECT
    s.name AS "Магазин",
    COUNT(i.id) AS "Кол-во инвентаризаций",
    SUM(i.actual_stock) AS "Общий учётный остаток"
FROM inventory i
JOIN store s ON i.store_id = s.id
GROUP BY s.name
HAVING COUNT(i.id) > 3
ORDER BY COUNT(i.id) DESC;
```



	Магазин	Кол-во инвентаризаций	Общий учётный остаток
1	Главный магазин	4	6 650

2.3. Комбинированный запрос: WHERE + GROUP BY + HAVING + ORDER BY

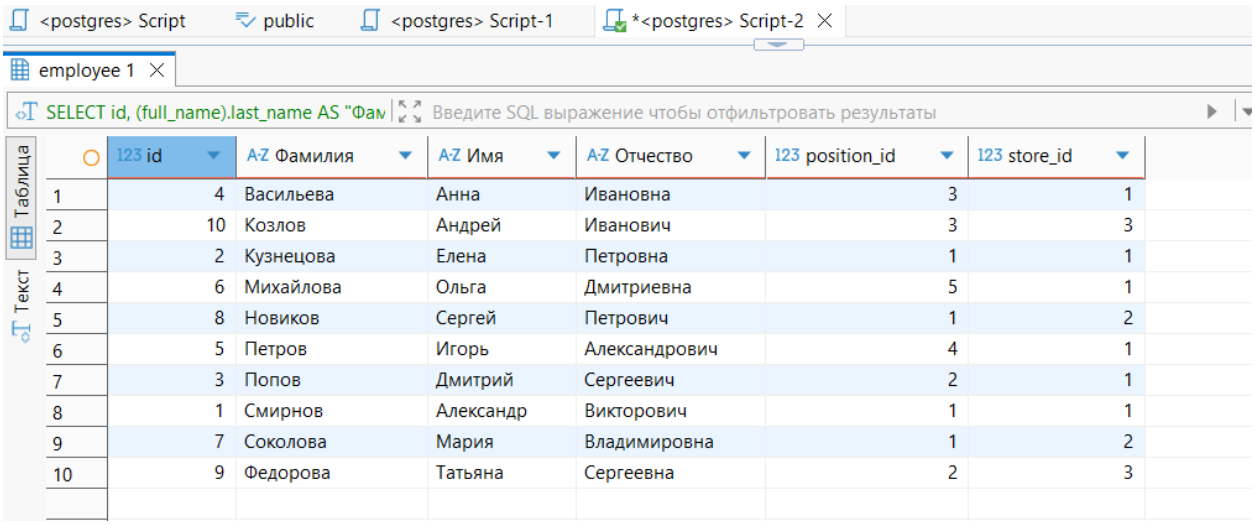
```sql
SELECT
    sup.name AS "Поставщик",
    COUNT(s.id) AS "Кол-во поставок",
    SUM(s.summa) AS "Общая сумма поставок"
FROM supply s
JOIN supplier sup ON s.supplier_id = sup.id
WHERE s.date >= '2024-01-01' AND s.date < '2025-01-01'
GROUP BY sup.name
HAVING SUM(s.summa) > 50000
ORDER BY SUM(s.summa) DESC;
```



	Поставщик	Кол-во поставок	Общая сумма поставок
1	ООО Соки России	2	98 000
2	ООО Напитки Плюс	2	83 000
3	ЗАО АлкоТрейд	2	74 000
4	ИП Иванов П.С.	2	63 000

2.4. Бонус: работа с составным типом fio_valid

```sql
SELECT
    id,
    (full_name).last_name AS "Фамилия",
    (full_name).first_name AS "Имя",
    (full_name).middle_name AS "Отчество",
    position_id,
    store_id
FROM employee
ORDER BY (full_name).last_name;
```



	id	Фамилия	Имя	Отчество	position_id	store_id
1	4	Васильева	Анна	Ивановна	3	1
2	10	Козлов	Андрей	Иванович	3	3
3	2	Кузнецова	Елена	Петровна	1	1
4	6	Михайлова	Ольга	Дмитриевна	5	1
5	8	Новиков	Сергей	Петрович	1	2
6	5	Петров	Игорь	Александрович	4	1
7	3	Попов	Дмитрий	Сергеевич	2	1
8	1	Смирнов	Александр	Викторович	1	1
9	7	Соколова	Мария	Владимировна	1	2
10	9	Федорова	Татьяна	Сергеевна	2	3

Задание №3

3.1. UPDATE №1 - точечное обновление

ДО:

```sql
SELECT
    name AS "Товар",
    retail_price AS "Новая цена",
    category_id
FROM product
WHERE category_id = (SELECT id FROM category WHERE name = 'Вода');
```

product 1

SELECT name AS "Товар", retail_price AS Введите SQL выражение чтобы отфильтровать результат

Таблица

	A-Z Товар	123 Новая цена	123 category_id
1	Вода Святой Источник 0.5л	72,48	3

После:

```sql
UPDATE product
SET retail_price = retail_price * 1.10
WHERE category_id = (SELECT id FROM category WHERE name = 'Вода');
```

```sql
SELECT
    name AS "Товар",
    retail_price AS "Новая цена",
    category_id
FROM product
WHERE category_id = (SELECT id FROM category WHERE name = 'Вода');
```

product 1

SELECT name AS "Товар", retail_price AS Введите SQL выражение чтобы отфильтровать резу

аблица

	A-Z Товар	123 Новая цена	123 category_id
1	Вода Святой Источник 0.5л	79,73	3

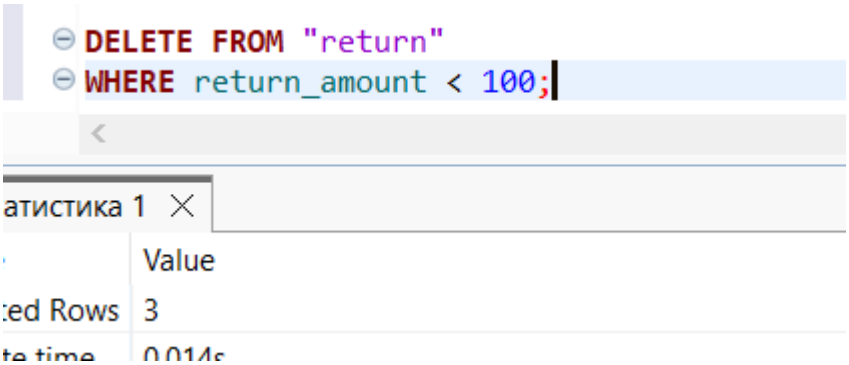
3.2. UPDATE №2 - массовое обновление

```sql
UPDATE stock
SET min_quantity = min_quantity + 10
WHERE product_id IN (SELECT id FROM product WHERE retail_price > 100);
```

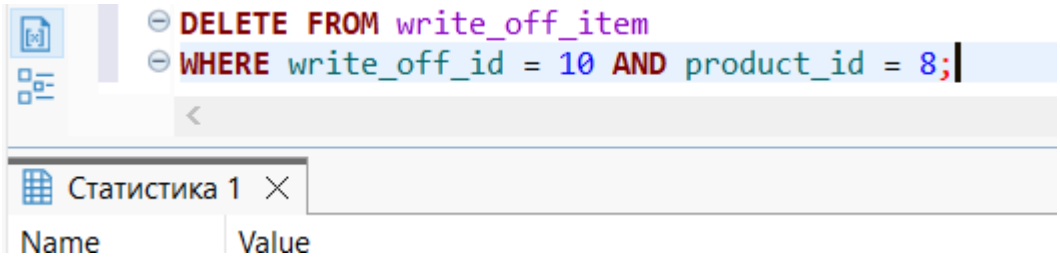
Статистика 1

Name	Value
Updated Rows	4

3.3. DELETE №1 - безопасное удаление по условию

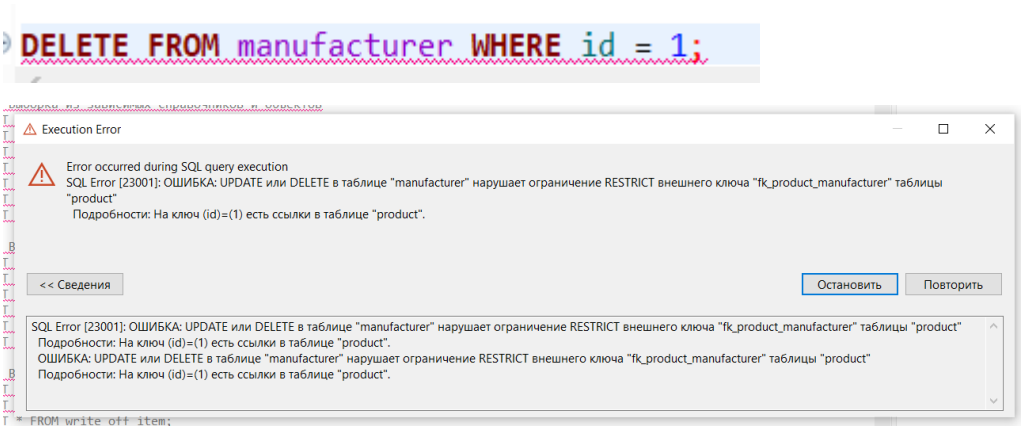


3.4. DELETE №2 - удаление конкретной записи



Задание №4

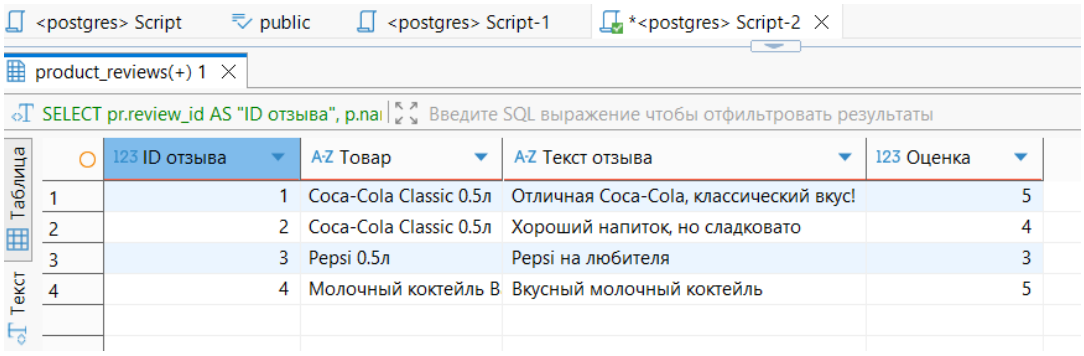
4.1. Срабатывание ON DELETE RESTRICT



4.2. Создание таблицы с ON DELETE CASCADE

```sql
CREATE TABLE product_reviews (
    review_id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    product_id INT NOT NULL,
    review_text TEXT NOT NULL,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    CONSTRAINT fk_review_product
        FOREIGN KEY (product_id)
        REFERENCES product(id)
        ON DELETE CASCADE
);

-- Наполняем тестовыми данными
INSERT INTO product_reviews (product_id, review_text, rating) VALUES
(1, 'Отличная Coca-Cola, классический вкус!', 5),
(1, 'Хороший напиток, но сладковато', 4),
(2, 'Pepsi на любителя', 3),
(12, 'Вкусный молочный коктейль', 5);
```



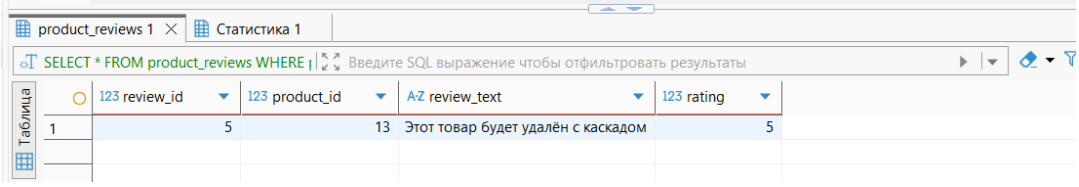
	ID отзыва	Товар	Текст отзыва	Оценка
1	1	Coca-Cola Classic 0.5л	Отличная Coca-Cola, классический вкус!	5
2	2	Coca-Cola Classic 0.5л	Хороший напиток, но сладковато	4
3	3	Pepsi 0.5л	Pepsi на любителя	3
4	4	Молочный коктейль В	Вкусный молочный коктейль	5

4.3. Демонстрация каскадного удаления

```sql
INSERT INTO product(name, category_id, manufacturer_id, unit_id, barcode, retail_price, expiration_date)
VALUES ('Тестовый товар для удаления', 1, 1, 1, '4680123456999', 50.00, '2027-01-01');

INSERT INTO product_reviews (product_id, review_text, rating)
VALUES (13, 'Этот товар будет удалён с каскадом', 5);

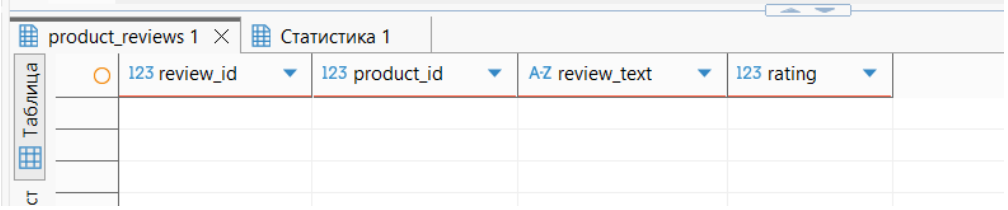
-- Проверяем, что отзыв добавился
SELECT * FROM product_reviews WHERE product_id = 13;
```



	review_id	product_id	review_text	rating
1	5	13	Этот товар будет удалён с каскадом	5

```sql
-- Удаляем тестовый товар - отзыв удалится автоматически (CASCADE)
DELETE FROM product WHERE id = 13;

-- Проверяем, что отзыв тоже удалился
SELECT * FROM product_reviews WHERE product_id = 13;
```



review_id	product_id	review_text	rating

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 2

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

Отчет представлен «19» сентября 2026 г.

Москва 2026 г.

Задание № 1 Демонстрация различных типов соединений

1.1 INNER JOIN (внутреннее соединение)

Задача: Вывести все позиции продаж с информацией о товарах (только те, где есть связь).

```
SELECT *
FROM sale_item si
INNER JOIN product p ON si.product_id = p.id;
```

sale_item(+) 1

SELECT * FROM sale_item si INNER JOIN p | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	123 sale_id	123 product_id	123 quantity	123 price	123 id	A-Z name	123 category_id
1	1	1	2	85	1	Coca-Cola Classic 0.5л	1
2	1	7	3	45	7	Вода Святой Источник	3
3	1	10	2	75	10	Lipton Ice Tea 0.5л	5
4	2	2	4	80	2	Pepsi 0.5л	1
5	3	3	3	82	3	Fanta Апельсин 0.5л	1
6	3	8	2	150	8	Red Bull 0.25л	4
7	3	12	1	95	12	Молочный коктейль В	7
8	4	7	2	45	7	Вода Святой Источник	3
9	4	10	1	75	10	Lipton Ice Tea 0.5л	5
10	5	5	3	120	5	Rich Апельсиновый 1л	2
11	5	6	2	115	6	Добрый Яблочный 1л	2
12	5	11	1	350	11	Jacobs Кофе в зернах	6
13	6	1	4	85	1	Coca-Cola Classic 0.5л	1
14	6	4	3	82	4	Sprite 0.5л	1
15	6	9	2	145	9	Monster Energy 0.5л	4
16	7	2	5	80	2	Pepsi 0.5л	1
17	7	3	4	82	3	Fanta Апельсин 0.5л	1
18	7	8	3	150	8	Red Bull 0.25л	4
19	8	7	4	45	7	Вода Святой Источник	3
20	8	10	2	75	10	Lipton Ice Tea 0.5л	5
21	9	1	3	85	1	Coca-Cola Classic 0.5л	1
22	9	5	2	120	5	Rich Апельсиновый 1л	2
23	9	12	2	95	12	Молочный коктейль В	7
24	10	6	4	115	6	Добрый Яблочный 1л	2
25	10	11	1	350	11	Jacobs Кофе в зернах	6

123 category_id	123 manufacturer_id	123 unit_id	A-Z barcode	123 retail_price	expiration_date
1	1	2	4680123456789	85	2027-06-15
3	3	2	4680123456795	79,73	2028-01-20
5	11	2	4680123456798	75	2027-04-10
1	2	2	4680123456790	80	2027-05-20
1	1	2	4680123456791	82	2027-07-10
4	9	2	4680123456796	150	2027-08-30
7	5	2	4680123456800	95	2026-10-05
3	3	2	4680123456795	79,73	2028-01-20
5	11	2	4680123456798	75	2027-04-10
2	7	3	4680123456793	120	2026-12-01
2	1	3	4680123456794	115	2026-11-15
6	12	4	4680123456799	350	2027-12-31
1	1	2	4680123456789	85	2027-06-15
1	1	2	4680123456792	82	2027-06-25
4	10	2	4680123456797	145	2027-09-15
1	2	2	4680123456790	80	2027-05-20
1	1	2	4680123456791	82	2027-07-10
4	9	2	4680123456796	150	2027-08-30
3	3	2	4680123456795	79,73	2028-01-20
5	11	2	4680123456798	75	2027-04-10
1	1	2	4680123456789	85	2027-06-15
2	7	3	4680123456793	120	2026-12-01
7	5	2	4680123456800	95	2026-10-05
2	1	3	4680123456794	115	2026-11-15
6	12	4	4680123456799	350	2027-12-31

Объяснение:

INNER JOIN возвращает только совпадающие строки из обеих таблиц

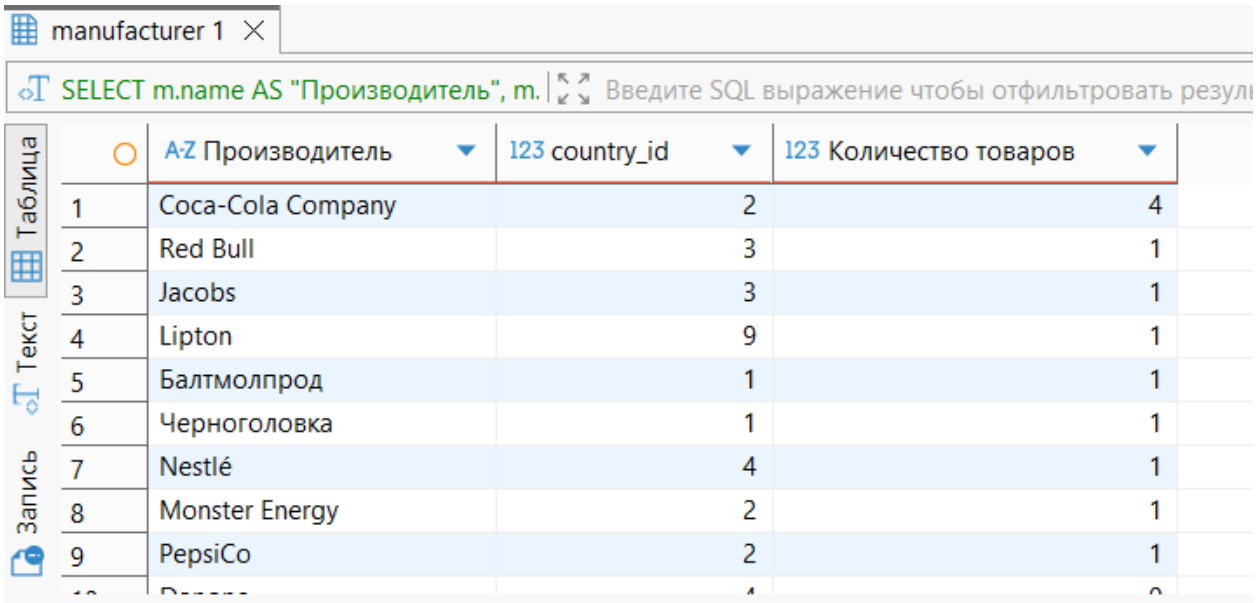
Если в sale_item есть запись с product_id, которого нет в product, она не попадёт в результат

SELECT * показывает все столбцы из обеих таблиц

1.2 LEFT JOIN (левое внешнее соединение)

Задача: Вывести всех производителей и количество товаров от каждого (включая тех, у кого товаров нет).

```sql
SELECT
    m.name AS "Производитель",
    m.country_id,
    COUNT(p.id) AS "Количество товаров"
FROM manufacturer m
LEFT JOIN product p ON m.id = p.manufacturer_id
GROUP BY m.id, m.name, m.country_id
ORDER BY "Количество товаров" DESC;
```



	Производитель	country_id	Количество товаров
1	Coca-Cola Company	2	4
2	Red Bull	3	1
3	Jacobs	3	1
4	Lipton	9	1
5	Балтмолпрод	1	1
6	Черноголовка	1	1
7	Nestlé	4	1
8	Monster Energy	2	1
9	PepsiCo	2	1

LEFT JOIN возвращает все строки из левой таблицы (manufacturer)

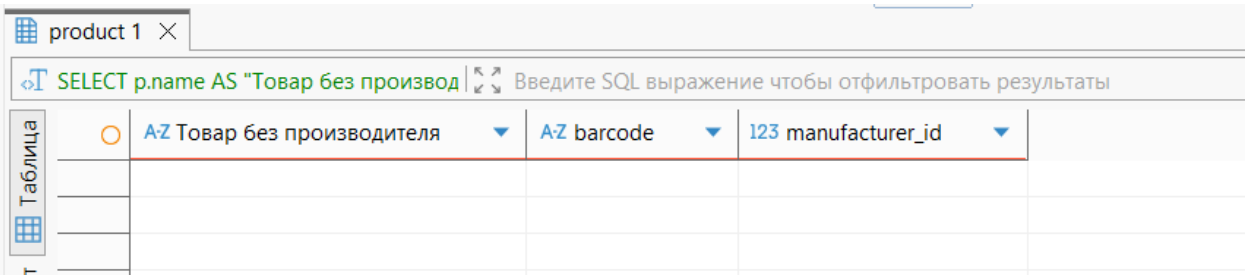
Если у производителя нет товаров, столбцы из product будут NULL

COUNT(p.id) считает только НЕ-NULL значения, поэтому для производителей без товаров вернётся 0

1.3 RIGHT JOIN + WHERE IS NULL (анти-соединение)

Задача: Найти товары, у которых нет производителя в базе (нарушение целостности).

```sql
SELECT
    p.name AS "Товар без производителя",
    p.barcode,
    p.manufacturer_id
FROM manufacturer m
RIGHT JOIN product p ON m.id = p.manufacturer_id
WHERE m.id IS NULL;
```



RIGHT JOIN возвращает все строки из правой таблицы (product)

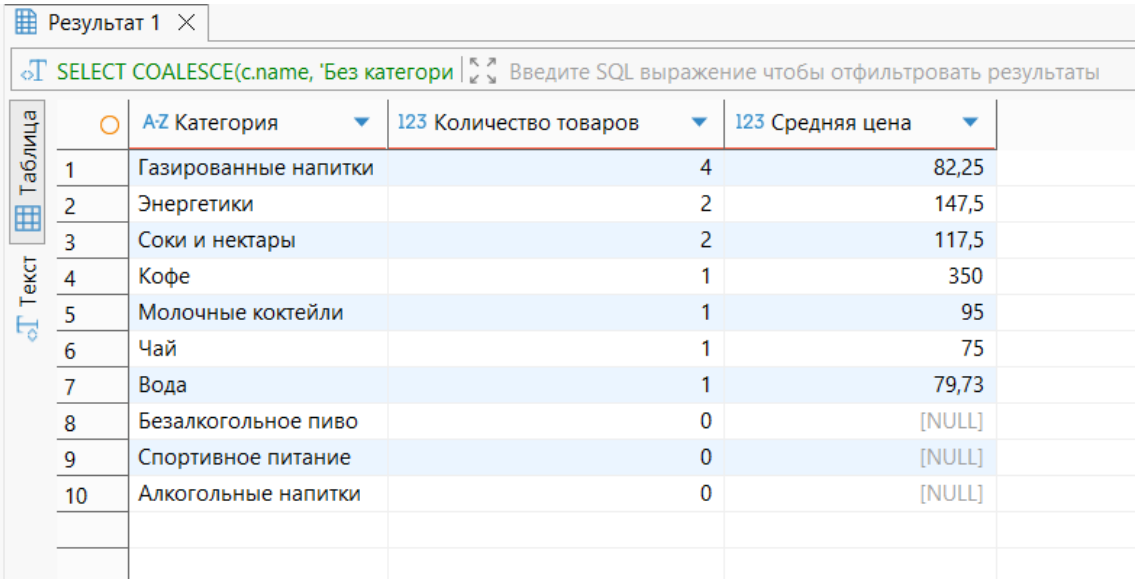
Для товаров без производителя поля из manufacturer будут NULL

WHERE m.id IS NULL отфильтровывает все успешные совпадения, оставляя только "осиротевшие" товары

1.4 FULL JOIN (полное внешнее соединение)

Задача: Проанализировать полную картину по категориям и товарам.

```sql
SELECT
    COALESCE(c.name, 'Без категории') AS "Категория",
    COUNT(p.id) AS "Количество товаров",
    ROUND(AVG(p.retail_price), 2) AS "Средняя цена"
FROM category c
FULL JOIN product p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY "Количество товаров" DESC;
```



	Категория	Количество товаров	Средняя цена
1	Газированные напитки	4	82,25
2	Энергетики	2	147,5
3	Соки и нектары	2	117,5
4	Кофе	1	350
5	Молочные коктейли	1	95
6	Чай	1	75
7	Вода	1	79,73
8	Безалкогольное пиво	0	[NULL]
9	Спортивное питание	0	[NULL]
10	Алкогольные напитки	0	[NULL]

FULL JOIN объединяет логику LEFT и RIGHT JOIN

Возвращает все строки из обеих таблиц

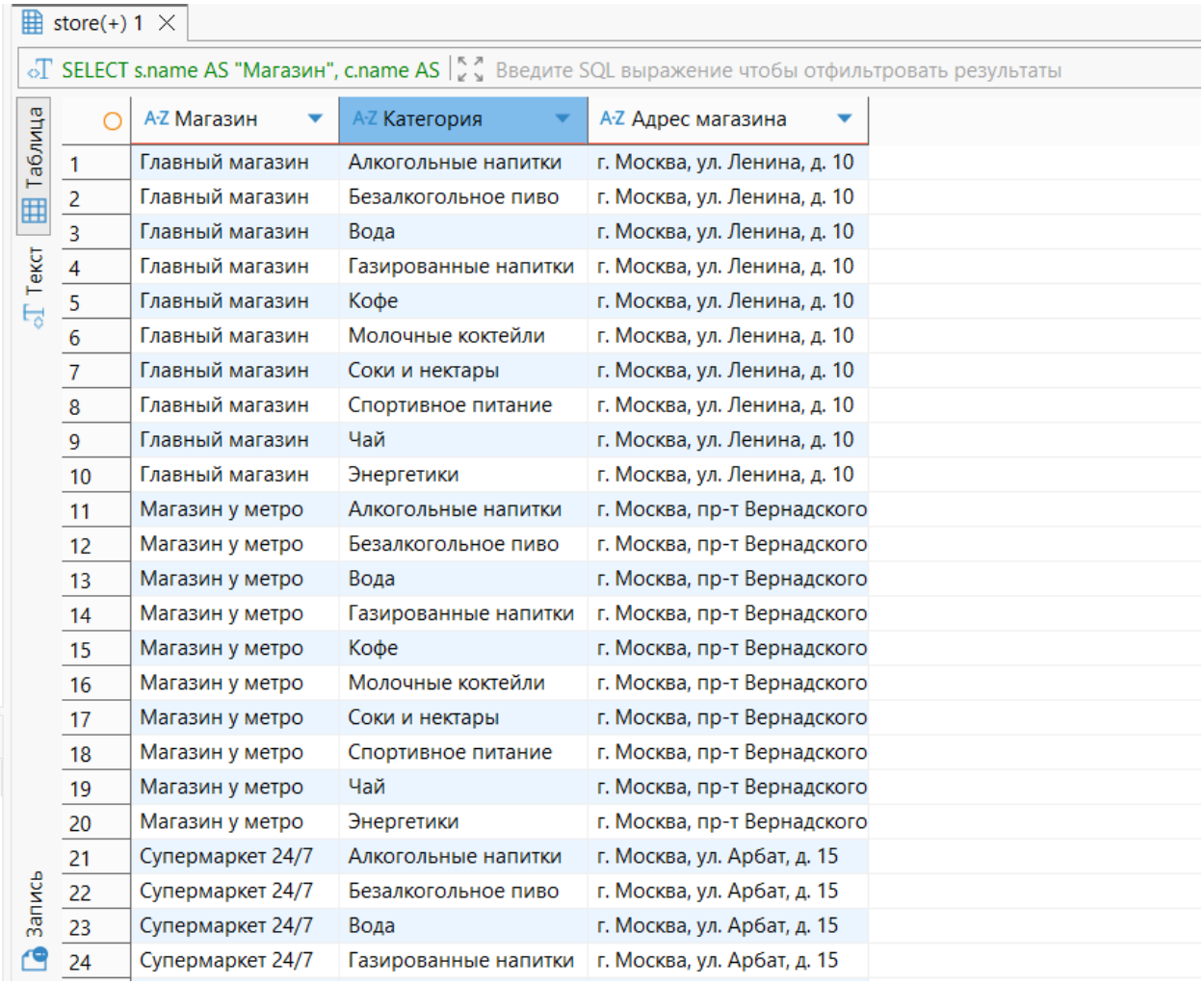
Если нет совпадения — недостающие поля заполняются NULL

COALESCE(c.name, 'Без категории') заменяет NULL на читаемый текст

1.5 CROSS JOIN (декартово произведение)

Задача: Сгенерировать все возможные комбинации магазинов и категорий товаров.

```sql
SELECT
    s.name AS "Магазин",
    c.name AS "Категория",
    s.address AS "Адрес магазина"
FROM store s
CROSS JOIN category c
ORDER BY s.name, c.name;
```


store(+) 1

SELECT s.name AS "Магазин", c.name AS Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	Магазин	Категория	Адрес магазина
1	Главный магазин	Алкогольные напитки	г. Москва, ул. Ленина, д. 10
2	Главный магазин	Безалкогольное пиво	г. Москва, ул. Ленина, д. 10
3	Главный магазин	Вода	г. Москва, ул. Ленина, д. 10
4	Главный магазин	Газированные напитки	г. Москва, ул. Ленина, д. 10
5	Главный магазин	Кофе	г. Москва, ул. Ленина, д. 10
6	Главный магазин	Молочные коктейли	г. Москва, ул. Ленина, д. 10
7	Главный магазин	Соки и нектары	г. Москва, ул. Ленина, д. 10
8	Главный магазин	Спортивное питание	г. Москва, ул. Ленина, д. 10
9	Главный магазин	Чай	г. Москва, ул. Ленина, д. 10
10	Главный магазин	Энергетики	г. Москва, ул. Ленина, д. 10
11	Магазин у метро	Алкогольные напитки	г. Москва, пр-т Вернадского
12	Магазин у метро	Безалкогольное пиво	г. Москва, пр-т Вернадского
13	Магазин у метро	Вода	г. Москва, пр-т Вернадского
14	Магазин у метро	Газированные напитки	г. Москва, пр-т Вернадского
15	Магазин у метро	Кофе	г. Москва, пр-т Вернадского
16	Магазин у метро	Молочные коктейли	г. Москва, пр-т Вернадского
17	Магазин у метро	Соки и нектары	г. Москва, пр-т Вернадского
18	Магазин у метро	Спортивное питание	г. Москва, пр-т Вернадского
19	Магазин у метро	Чай	г. Москва, пр-т Вернадского
20	Магазин у метро	Энергетики	г. Москва, пр-т Вернадского
21	Супермаркет 24/7	Алкогольные напитки	г. Москва, ул. Арбат, д. 15
22	Супермаркет 24/7	Безалкогольное пиво	г. Москва, ул. Арбат, д. 15
23	Супермаркет 24/7	Вода	г. Москва, ул. Арбат, д. 15
24	Супермаркет 24/7	Газированные напитки	г. Москва, ул. Арбат, д. 15

CROSS JOIN не имеет условия ON - соединяет каждую строку первой таблицы с каждой строкой второй

Формула: если в store 5 строк, а в category 10 строк - результат: 5 × 10 = 50 строк

1.6 SELF-JOIN (само-соединение)

Задача: Создать иерархию сотрудников (показать руководителя для каждого сотрудника).

SELF-JOIN - это соединение таблицы с самой собой

e (employee) - роль "сотрудника"

m (manager) - та же таблица, но в роли "руководителя"

LEFT JOIN используется, чтобы показать и сотрудников без руководителя (у кого manager_id IS NULL)

ON e.manager_id = m.id - находим руководителя по его ID

```sql
-- Добавляем столбец manager_id для иерархии
ALTER TABLE employee
ADD COLUMN manager_id INT;

-- Добавляем внешний ключ (ссылка на самого себя)
ALTER TABLE employee
ADD CONSTRAINT fk_employee_manager
FOREIGN KEY (manager_id)
REFERENCES employee(id)
ON DELETE SET NULL;

-- Устанавливаем иерархию (управляющий — руководитель для остальных)
UPDATE employee
SET manager_id = 6  -- id=6 это Михайлова О.Д. (Управляющий)
WHERE position_id != 5 AND id != 6;

-- У администраторов руководитель — товаровед (id=5)
UPDATE employee
SET manager_id = 5
WHERE position_id = 3;

-- Показывает сотрудника и его руководителя
SELECT
    e.full_name AS "Сотрудник",
    e.position_id AS "Должность сотрудника",
    m.full_name AS "Руководитель",
    m.position_id AS "Должность руководителя"
FROM employee e
LEFT JOIN employee m ON e.manager_id = m.id
ORDER BY e.id;
```

employee 1 | Статистика 1

SELECT e.full_name AS "Сотрудник", e.po | Введите SQL выражение чтобы отфильтровать результаты

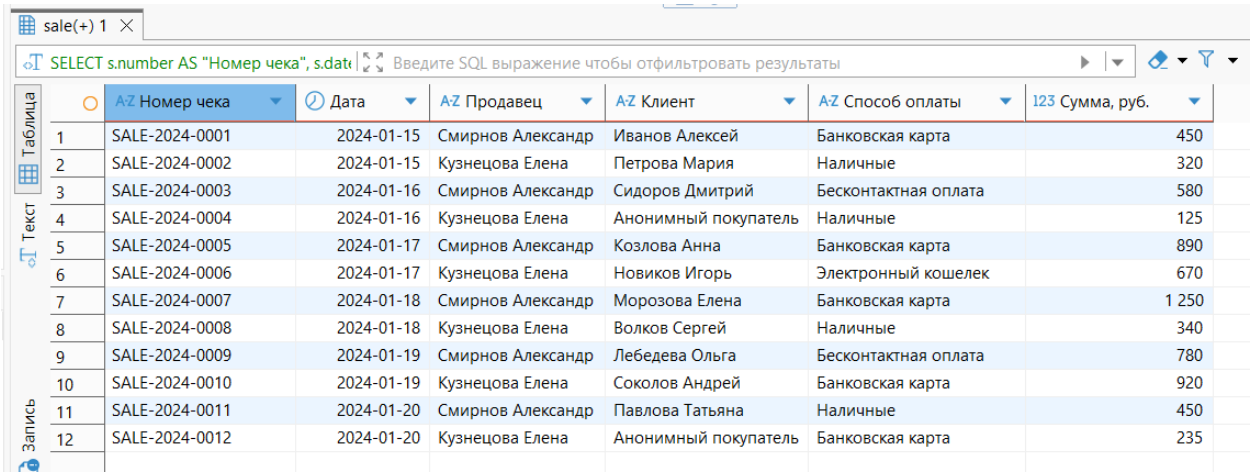
	Сотрудник			Должность сотрудника	Руководитель	
	last_name	first_name	middle_name		last_name	first_name
1	Смирнов	Александр	Викторович	1	Михайлова	Ольга
2	Кузнецова	Елена	Петровна	1	Михайлова	Ольга
3	Попов	Дмитрий	Сергеевич	2	Михайлова	Ольга
4	Васильева	Анна	Ивановна	3	Петров	Игорь
5	Петров	Игорь	Александрович	4	Михайлова	Ольга
6	Михайлова	Ольга	Дмитриевна	5	[NULL]	[NULL]
7	Соколова	Мария	Владимировна	1	Михайлова	Ольга
8	Новиков	Сергей	Петрович	1	Михайлова	Ольга
9	Федорова	Татьяна	Сергеевна	2	Михайлова	Ольга
10	Козлов	Андрей	Иванович	3	Петров	Игорь

middle_name	Должность руководителя
Дмитриевна	5
Дмитриевна	5
Дмитриевна	5
Александрович	4
Дмитриевна	5
[NULL]	[NULL]
Дмитриевна	5
Дмитриевна	5
Дмитриевна	5
Александрович	4

1.7 Сложный запрос с 3+ таблицами

Задача: Вывести полную информацию о продажах: номер чека, дата, ФИО продавца, ФИО клиента, способ оплаты, общая сумма.

```sql
-- Полная информация о каждой продаже
SELECT
    s.number AS "Номер чека",
    s.date::DATE AS "Дата",
    (e.full_name).last_name || ' ' ||
    (e.full_name).first_name AS "Продавец",
    COALESCE(
        (c.full_name).last_name || ' ' || (c.full_name).first_name,
        'Анонимный покупатель'
    ) AS "Клиент",
    pt.name AS "Способ оплаты",
    s.total_amount AS "Сумма, руб."
FROM sale s
JOIN employee e ON s.employee_id = e.id
LEFT JOIN client c ON s.client_id = c.id
JOIN payment_type pt ON s.payment_type_id = pt.id
ORDER BY s.date;
```



sale(+) 1

SELECT s.number AS "Номер чека", s.date Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	Номер чека	Дата	Продавец	Клиент	Способ оплаты	Сумма, руб.
1	SALE-2024-0001	2024-01-15	Смирнов Александр	Иванов Алексей	Банковская карта	450
2	SALE-2024-0002	2024-01-15	Кузнецова Елена	Петрова Мария	Наличные	320
3	SALE-2024-0003	2024-01-16	Смирнов Александр	Сидоров Дмитрий	Бесконтактная оплата	580
4	SALE-2024-0004	2024-01-16	Кузнецова Елена	Анонимный покупатель	Наличные	125
5	SALE-2024-0005	2024-01-17	Смирнов Александр	Козлова Анна	Банковская карта	890
6	SALE-2024-0006	2024-01-17	Кузнецова Елена	Новиков Игорь	Электронный кошелек	670
7	SALE-2024-0007	2024-01-18	Смирнов Александр	Морозова Елена	Банковская карта	1 250
8	SALE-2024-0008	2024-01-18	Кузнецова Елена	Волков Сергей	Наличные	340
9	SALE-2024-0009	2024-01-19	Смирнов Александр	Лебедева Ольга	Бесконтактная оплата	780
10	SALE-2024-0010	2024-01-19	Кузнецова Елена	Соколов Андрей	Банковская карта	920
11	SALE-2024-0011	2024-01-20	Смирнов Александр	Павлова Татьяна	Наличные	450
12	SALE-2024-0012	2024-01-20	Кузнецова Елена	Анонимный покупатель	Банковская карта	235

4 таблицы соединяются последовательно:

sale (шапка продажи)

employee (кто продал) - через s.employee_id = e.id

client (кто купил) - через s.client_id = c.id

payment_type (как оплатили) - через s.payment_type_id = pt.id

Логика выполнения СУБД:

Сначала соединяет sale + employee - промежуточный результат

Затем к нему присоединяет client - новый промежуточный результат

Наконец, присоединяет payment_type - финальный результат

LEFT JOIN для client - потому что могут быть анонимные продажи (client_id IS NULL)

COALESCE(...) - заменяет NULL (для анонимных покупок) на текст 'Анонимный покупатель'

(e.full_name).last_name - извлечение полей из составного типа fio_valid

Задание №2 Применение теоретико-множественных операторов

2.1 UNION (объединение множеств)

Задача: Создать единый список всех уникальных названий (товары + категории + производители).

```
-- Единый список всех наименований
SELECT name AS "Наименование", 'Товар' AS "Тип"
FROM product
UNION
SELECT name AS "Наименование", 'Категория' AS "Тип"
FROM category
UNION
SELECT name AS "Наименование", 'Производитель' AS "Тип"
FROM manufacturer
ORDER BY "Наименование";
```

Результат 1

SELECT name AS "Наименование", 'Това | Введите SQL вы

Таблица | Текст | Запись

	Наименование	Тип
1	Coca-Cola Classic 0.5л	Товар
2	Coca-Cola Company	Производитель
3	Danone	Производитель
4	Fanta Апельсин 0.5л	Товар
5	Jacobs	Производитель
6	Jacobs Кофе в зернах 250г	Товар
7	Lipton	Производитель
8	Lipton Ice Tea 0.5л	Товар
9	Monster Energy	Производитель
10	Monster Energy 0.5л	Товар
11	Nestlé	Производитель
12	Pepsi 0.5л	Товар
13	PepsiCo	Производитель

UNION объединяет результаты нескольких запросов по вертикали (добавляет строки)

Главное правило: все запросы должны возвращать одинаковое количество столбцов с совместимыми типами

UNION автоматически удаляет дубликаты (если название совпадает в разных таблицах, оно появится один раз)

2.2 INTERSECT (пересечение множеств)

Задача: Найти товары, которые есть в наличии И которые были проданы.

```sql
-- Товары, которые И есть в ассортименте, И были проданы
SELECT p.name AS "Товар"
FROM product p
INTERSECT
SELECT p2.name AS "Товар"
FROM product p2
INNER JOIN sale_item si ON p2.id = si.product_id
ORDER BY "Товар";
```

Результат 1

SELECT p.name AS "Товар" FROM produc | Введит

Таблица | Текст

	Товар
1	Coca-Cola Classic 0.5л
2	Fanta Апельсин 0.5л
3	Jacobs Кофе в зернах 250г
4	Lipton Ice Tea 0.5л
5	Monster Energy 0.5л
6	Pepsi 0.5л
7	Red Bull 0.25л
8	Rich Апельсиновый 1л
9	Sprite 0.5л
10	Вода Святой Источник 0.5л
11	Добрый Яблочный 1л
12	Молочный коктейль Ванил

INTERSECT возвращает только те строки, которые есть в обоих запросах одновременно

Первый запрос: все товары из таблицы product

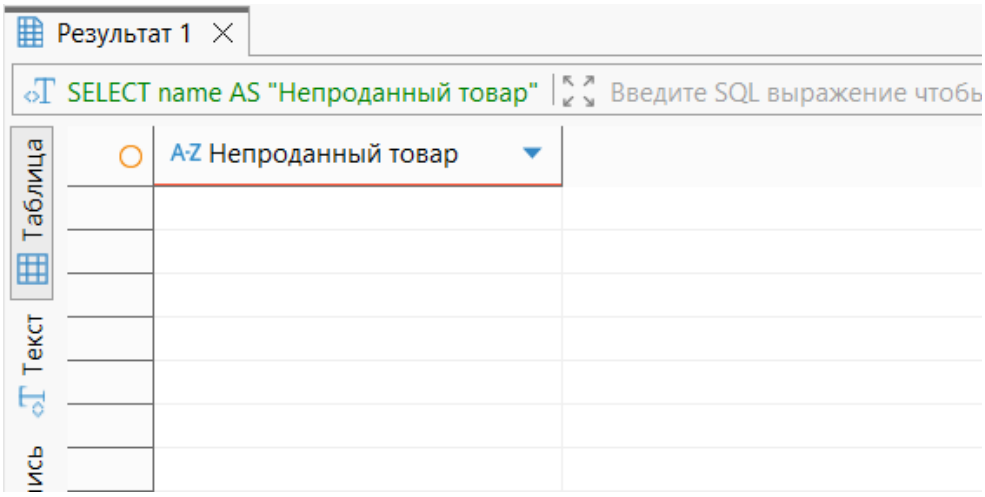
Второй запрос: только те товары, которые есть в sale_item (т.е. были проданы)

Результат: товары, которые пересекаются в обоих множествах

2.3 EXCEPT (разность множеств)

Задача: Найти товары, которые есть в ассортименте, но которые НИ РАЗУ не продавались.

```sql
-- Товары, которые есть в наличии, но не были проданы
SELECT name AS "Непроданный товар"
FROM product
EXCEPT
SELECT p.name AS "Непроданный товар"
FROM product p
INNER JOIN sale_item si ON p.id = si.product_id
ORDER BY "Непроданный товар";
```



EXCEPT возвращает строки из первого запроса, которых НЕТ во втором запросе

Первый запрос: все товары

Второй запрос: проданные товары

Результат: товары без продаж (разность множеств)